Day 13 - Database Fundamentals

Student Management Database Design and ER Diagram (MySQL)

1. Database fundamentals (quick revision)

- **Database:** an organised collection of data stored so it can be easily searched, updated and managed.
- **Relational database:** stores data in tables that are linked to each other through keys (e.g. PostgreSQL, MySQL).
- **Table, row, column:** a table is an entity (Students); a row is one record (one student); a column is one attribute (email).
- **Primary key (PK):** uniquely identifies each row; cannot be NULL or duplicated.
- **Foreign key (FK):** a column that refers to the primary key of another table and keeps data consistent.
- **Relationships:** one-to-one (1:1), one-to-many (1:N) and many-to-many (M:N). M:N is resolved with a junction table.
- **ER diagram:** a picture of the entities (tables), their attributes and the relationships between them.

2. Entities and fields

The database is built in **MySQL** and has five tables: departments, students, instructors, courses and enrollments.

departments

Field	Data type	Key / constraint
department_id	INT AUTO_INCREMENT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL, UNIQUE
head_name	VARCHAR(100)	Optional

students

Field	Data type	Key / constraint
student_id	INT AUTO_INCREMENT	PRIMARY KEY
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
phone	VARCHAR(20)	Optional
date_of_birth	DATE	Optional
gender	VARCHAR(10)	Optional
enrollment_date	DATE	DEFAULT (CURRENT_DATE)
department_id	INT	FOREIGN KEY -> departments

instructors

Field	Data type	Key / constraint
instructor_id	INT AUTO_INCREMENT	PRIMARY KEY
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL

Field	Data type	Key / constraint
email	VARCHAR(100)	NOT NULL, UNIQUE
department_id	INT	FOREIGN KEY -> departments

courses

Field	Data type	Key / constraint
course_id	INT AUTO_INCREMENT	PRIMARY KEY
course_code	VARCHAR(10)	NOT NULL, UNIQUE
title	VARCHAR(100)	NOT NULL
credits	INT	CHECK (credits > 0)
department_id	INT	FOREIGN KEY -> departments
instructor_id	INT	FOREIGN KEY -> instructors

enrollments (junction table)

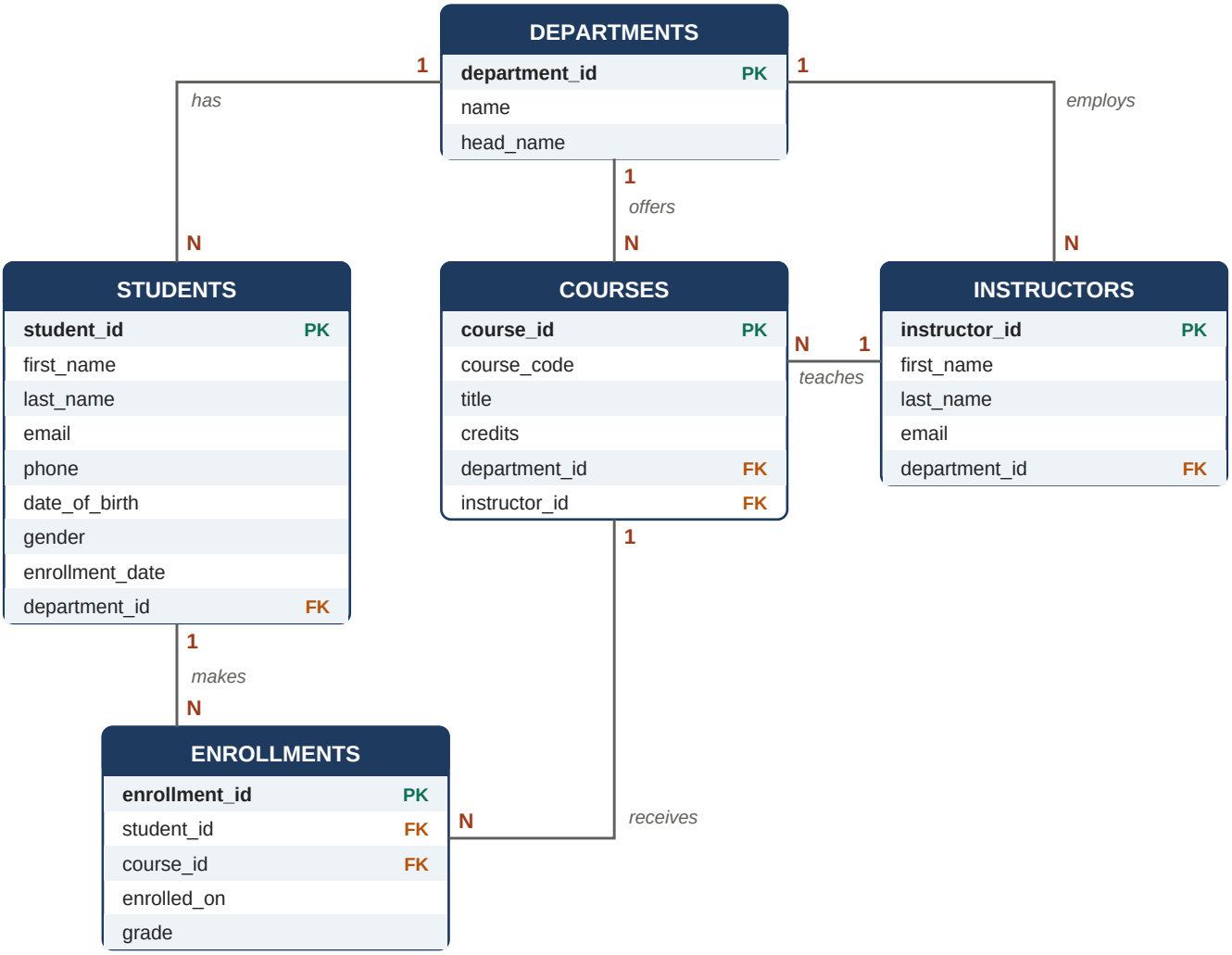
Field	Data type	Key / constraint
enrollment_id	INT AUTO_INCREMENT	PRIMARY KEY
student_id	INT	FOREIGN KEY -> students, ON DELETE CASCADE
course_id	INT	FOREIGN KEY -> courses, ON DELETE CASCADE
enrolled_on	DATE	DEFAULT (CURRENT_DATE)
grade	VARCHAR(2)	Optional
(student_id, course_id)	-	UNIQUE pair: no duplicate enrollment

3. Relationships

Relationship	Type	Meaning
departments - students	1 : N	One department has many students
departments - instructors	1 : N	One department employs many instructors
departments - courses	1 : N	One department offers many courses
instructors - courses	1 : N	One instructor teaches many courses
students - courses	M : N	A student takes many courses and a course has many students; resolved by enrollm

4. ER diagram

PK = primary key, FK = foreign key. The labels 1 and N at the ends of each line show the cardinality (one / many).



5. SQL script (MySQL)

```
CREATE DATABASE student_management;
USE student_management;

CREATE TABLE departments (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL UNIQUE,
    head_name     VARCHAR(100)
);

CREATE TABLE students (
    student_id      INT AUTO_INCREMENT PRIMARY KEY,
    first_name      VARCHAR(50)  NOT NULL,
    last_name       VARCHAR(50)  NOT NULL,
    email           VARCHAR(100) NOT NULL UNIQUE,
    phone           VARCHAR(20),
    date_of_birth   DATE,
    gender          VARCHAR(10),
    enrollment_date DATE DEFAULT (CURRENT_DATE),
    department_id   INT,
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

CREATE TABLE instructors (
    instructor_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name    VARCHAR(50)  NOT NULL,
    last_name     VARCHAR(50)  NOT NULL,
    email         VARCHAR(100) NOT NULL UNIQUE,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

CREATE TABLE courses (
    course_id     INT AUTO_INCREMENT PRIMARY KEY,
    course_code   VARCHAR(10)  NOT NULL UNIQUE,
    title         VARCHAR(100) NOT NULL,
    credits       INT CHECK (credits > 0),
    department_id INT,
    instructor_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(department_id),
    FOREIGN KEY (instructor_id) REFERENCES instructors(instructor_id)
);

CREATE TABLE enrollments (
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id    INT NOT NULL,
    course_id     INT NOT NULL,
    enrolled_on   DATE DEFAULT (CURRENT_DATE),
    grade         VARCHAR(2),
    UNIQUE (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)   ON DELETE CASCADE
);
```

6. Sample data and test query

```
INSERT INTO departments (name, head_name) VALUES ('Computer Science', 'Dr. Rao');
INSERT INTO instructors (first_name, last_name, email, department_id)
VALUES ('Anita', 'Joshi', 'anita@college.edu', 1);
INSERT INTO courses (course_code, title, credits, department_id, instructor_id)
VALUES ('CS101', 'Intro to Programming', 4, 1, 1);
INSERT INTO students (first_name, last_name, email, department_id)
VALUES ('Rahul', 'Patil', 'rahul@college.edu', 1);
INSERT INTO enrollments (student_id, course_id) VALUES (1, 1);

SELECT s.first_name, c.title, i.last_name AS instructor
FROM enrollments e
JOIN students s    ON s.student_id = e.student_id
JOIN courses c     ON c.course_id = e.course_id
JOIN instructors i ON i.instructor_id = c.instructor_id;
```

7. Installing MySQL

- **Windows:** download MySQL Installer from dev.mysql.com/downloads/installer, choose *Developer Default* (installs MySQL Server and MySQL Workbench), and set a root password.
- **macOS:** `brew install mysql`, then `brew services start mysql`.
- **Ubuntu:** `sudo apt install mysql-server`, then `sudo mysql`.
- **Run the script:** open MySQL Workbench, connect to localhost, paste the script from section 5 and click the lightning (Execute) button. Or use `mysql -u root -p` in the terminal.
- **Verify:** run `SHOW TABLES;` - you should see 5 tables.
- **No install:** use db-fiddle.com and select MySQL.

8. Next step for Java full stack

Later, Spring Boot connects to this database through `application.properties`. Each table becomes an @Entity class, and the foreign keys become @OneToMany / @ManyToMany mappings.

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_management
spring.datasource.username=root
spring.datasource.password=your_password
spring.jpa.hibernate.ddl-auto=none
```